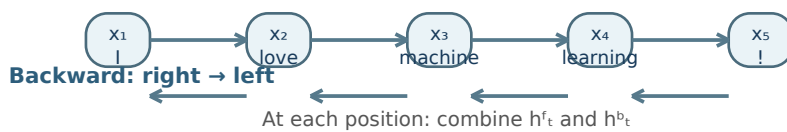


Bi-directional RNN — Deep Learning Notes

A visual, step-by-step guide to BiRNN architecture, TensorFlow/Keras parameters, tensor shapes, token-level prediction, training, and TimeDistributed.

Forward: left → right

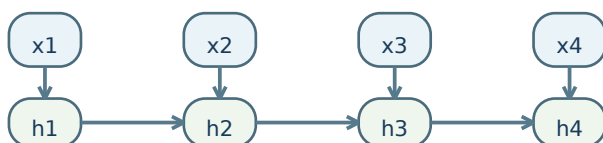


The central idea: a BiRNN reads the same sequence in two directions. At each timestep, the forward and backward representations are combined. If the RNN returns all timesteps, a classifier such as `Dense(5)` can produce one class prediction for every word.

This version intentionally uses flow diagrams, whitespace, and clean vector shapes rather than dense black-box layouts.

1. Start from the RNN you already understand

For a sentence such as “I love machine learning”, an RNN reads one word at a time. At timestep t it receives the current input x_t and the previous hidden state h_{t-1} . It produces a new hidden state h_t .



Each h_t summarizes the current input plus relevant context.

A useful mental model is: the hidden state is the RNN’s running summary. It is not simply a copy of the current word. It is a learned representation influenced by the sequence seen so far.

$$h_t = \text{activation}(W_x x_t + W_h h_{t-1} + b)$$

x_t = input representation at timestep t

h_{t-1} = previous hidden state

h_t = new hidden state

W_x, W_h = learned weights

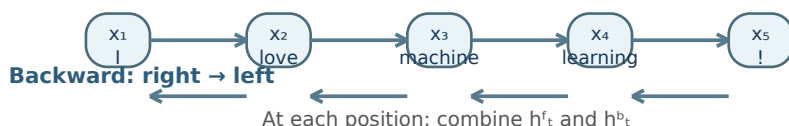
b = learned bias

When the RNN is used for a complete sentence, the final hidden state can summarize the sequence. But for token-level tasks, we often need every timestep’s hidden representation, which is exactly why `return_sequences=True` matters.

2. Why a Bidirectional RNN?

A forward RNN only has access to the past/current part of the sequence. For some language tasks, the meaning of a word becomes clearer when you also know what comes after it.

Forward: left → right



Example: in “I visited New York yesterday”, recognizing “New York” as a location can benefit from both left context and right context. The backward RNN provides information that flows from the later words toward the earlier positions.

Use BiRNN when future context is legitimately available. Typical examples include Named Entity Recognition (NER), POS tagging, sequence labeling, and other offline sequence-understanding tasks. A standard BiRNN is not causal, so it is generally unsuitable for real-time next-token generation where future

tokens are unknown.

3. What exactly happens at one word?

At timestep t there are two hidden representations:

```
Forward direction:  
h_t^forward = RNN_forward(x_t, h_{t-1}^forward)
```

```
Backward direction:  
h_t^backward = RNN_backward(x_t, h_{t+1}^backward)
```

```
With merge_mode="concat":  
h_t = [h_t^forward ; h_t^backward]
```

If each direction has 64 units, each hidden vector contains 64 values. Concatenation gives $64 + 64 = 128$ values per word.

This does not mean there is one giant RNN with 128 units. There are two recurrent networks, each with 64 units, and their outputs are combined.

4. Tensor shapes: the most important skill

For NLP, always ask: How many examples? How many timesteps? How many features per timestep?



Suppose a batch contains 32 sentences, each padded/truncated to 10 word positions, and each word is represented by a 128-dimensional embedding.

```
Input word IDs:  
X.shape = (32, 10)
```

```
After Embedding(output_dim=128):  
(32, 10, 128)
```

```
BiRNN with 64 units in each direction + concat:  
(32, 10, 128)
```

```
Dense(5, softmax):  
(32, 10, 5)
```

Interpretation of the final output: 32 sentences $\times$ 10 word positions $\times$ 5 class probabilities.

5. input_shape versus batch_size

This distinction causes a lot of confusion. The RNN layer's input shape describes one sample's sequence dimensions, not how many samples are in a training batch.

```
SimpleRNN(64, input_shape=(10, 128))
```

10 = sequence length

128 = features per timestep

The batch dimension is normally left flexible.

Actual training batch:
(32, 10, 128)

Keras conceptually treats this as:
(batch, 10, 128)

So batch_size=32 belongs to the training procedure (`fit`), while `input\_shape=(10, 128)` describes the shape of each example after embedding/features are available.

6. Batch size in sentence + embedding language

One sentence:
"I love machine learning"

4 words × 8 embedding values
shape = (4, 8)

32 sentences:
shape = (32, 4, 8)

Therefore:
32 = batch size
4 = sequence length
8 = embedding dimension

Before embedding, the same batch might be `(32, 4)` because each word is represented only by an integer ID. After embedding, each ID becomes a vector, creating the extra embedding dimension.

7. Embedding layer

Embedding(input_dim=10000, output_dim=128)

input_dim is the size of the vocabulary/index space. output_dim is the size of the learned vector for each token.

Word IDs:
(32, 10)
↓
Embedding(10000, 128)
↓
(32, 10, 128)

One word ID, e.g. 143:
143 → [128 learned values]

The embedding matrix itself is a learned parameter. During training, the model updates the vectors so that useful representations emerge for the task.

8. RNN parameters you should know

Parameter	What it controls	Typical mental question
units	Hidden-state size per direction	How many values represent h_t ?
return_sequences	Whether every timestep is returned	Do I need one output per word?
return_state	Whether final recurrent state is returned separately	Do I need explicit final state access?
activation	Nonlinear transformation in the recurrent update	How is the new hidden state transformed?
dropout	Regularization on input connections during training	How do I reduce overfitting?
recurrent_dropout	Regularization on recurrent connections	How do I regularize hidden-to-hidden connections?

9. return_sequences — the key to your NER question

If `return_sequences=False`, the recurrent layer returns one sequence-level representation. If `return_sequences=True`, it returns the representation at every timestep.

```
return_sequences=False:  
BiRNN output → (batch, 128)
```

```
return_sequences=True:  
BiRNN output → (batch, sequence_length, 128)
```

For NER, we normally want the second case because every word needs its own prediction.

10. Why Dense(5) gives five values for every word



This is exactly the concept you were asking about. With `return_sequences=True`, the BiRNN output contains a vector for every word. `Dense(5)` operates on the last dimension, transforming each 128-dimensional word representation into 5 class scores/probabilities.

```
BiRNN:  
(batch, 10, 128)
```

```
Dense(5):  
(batch, 10, 5)
```

```
For one sentence:  
word 1 → 5 probabilities  
word 2 → 5 probabilities  
...  
word 10 → 5 probabilities
```

So if your five NER classes were `'O, PERSON, LOCATION, ORGANIZATION, DATE'`, the five values for each word are the model's scores/probabilities for those five classes.

11. Do I need TimeDistributed(Dense(...))?

For a normal Keras Dense layer, not necessarily. Dense already applies its transformation to the final dimension of an N-dimensional tensor.

```
BiRNN output:  
(batch, sequence_length, 128)
```

```
Dense(5):  
(batch, sequence_length, 5)
```

This is enough for per-timestep classification.

You may also see:

```
TimeDistributed(Dense(5, activation="softmax"))
```

Conceptually, `TimeDistributed` explicitly says “apply this layer independently at each timestep.” For a plain Dense classifier on a 3-D RNN output, it is generally redundant. The important ingredient for getting one prediction per word is `return_sequences=True`, not `TimeDistributed`.

12. return_state

When `return_state=True`, the RNN exposes its final recurrent state separately. This is useful when you explicitly need the final state for another computation or for advanced encoder/decoder-style architectures.

```
output, state = rnn(x)
```

Do not confuse `return_sequences` and `return_state`: the first controls whether you get outputs across all timesteps; the second controls whether the final recurrent state is also returned.

13. Bidirectional merge_mode

```
Forward = [a, b, c]
Backward = [d, e, f]
```

```
concat → [a, b, c, d, e, f] → size 2H
sum → [a+d, b+e, c+f] → size H
ave → element-wise average → size H
mul → element-wise product → size H
```

For learning and most standard BiRNN setups, `concat` is the easiest to reason about because it preserves both directional representations.

14. Token-level versus sentence-level prediction

This is the most useful way to decide whether you need `return_sequences=True`.

Task	RNN output	Dense output	Sparse y
NER / POS / token tagging	(B, T, 2H)	(B, T, C)	(B, T)
Sentence classification	(B, 2H)	(B, C)	(B,)

Here B = batch size, T = sequence length, H = units in each direction, and C = number of classes.

15. What exactly is y for NER?

Suppose the model has 5 classes:

```
0 = 0
1 = PERSON
2 = LOCATION
3 = ORGANIZATION
4 = DATE
```

For the sentence “John lives in Delhi today”, the labels might be:

```
John → PERSON → 1
lives → 0 → 0
in → 0 → 0
Delhi → LOCATION → 2
today → DATE → 4
```

```
y = [1, 0, 0, 2, 4]
```

For 32 padded sentences of length 10:

```
y.shape = (32, 10)
```

```
Model prediction shape:
(32, 10, 5)
```

The model produces five probabilities per word; `y` contains the single correct class ID for each word when using `sparse_categorical_crossentropy`.

16. How loss compares prediction with y

For one word:

Prediction:

[0.02, 0.93, 0.01, 0.03, 0.01]

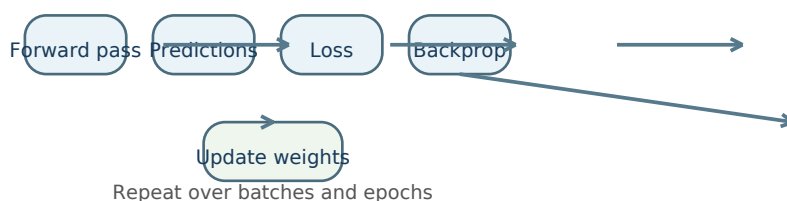
Actual sparse y:

1 # PERSON

The loss evaluates how much probability the model assigned to the correct class.

During training, the loss is aggregated over the relevant examples/timesteps, gradients are calculated, and the model parameters are updated.

17. Complete training flow



Training is not “run the model once and store the hidden states.” The forward and backward RNNs, embedding matrix, and Dense layer all contain trainable parameters that are updated through backpropagation.

Forward pass:

X → Embedding → Forward RNN + Backward RNN
→ merged BiRNN representations → Dense → predictions

Then:

predictions + y → loss

loss → gradients

gradients → update trainable weights

18. Complete TensorFlow/Keras example for NER

```
import tensorflow as tf
from tensorflow import keras
```

```
model = keras.Sequential([
    keras.layers.Embedding(
        input_dim=10000,
        output_dim=128
    ),
```

```
    keras.layers.Bidirectional(
        keras.layers.SimpleRNN(
            units=64,
            return_sequences=True,
            activation="tanh"
        ),
        merge_mode="concat"
    ),
```

```
    keras.layers.Dense(
        units=5,
```

```

activation="softmax"
)
])

model.compile(
optimizer=keras.optimizers.Adam(learning_rate=0.001),
loss="sparse_categorical_crossentropy",
metrics=["accuracy"]
)

model.fit(
X_train,
y_train,
batch_size=32,
epochs=20,
shuffle=True
)

```

For this model, if `X_train` is `(N, 10)` and there are 5 classes, the model's prediction tensor is `(N, 10, 5)` and `y_train` is `(N, 10)` for sparse integer labels (assuming the padding/masking strategy is handled appropriately).

19. What gets learned?

Component	Trainable information
Embedding	Vector representation for each token/index
Forward RNN	Input-to-hidden, hidden-to-hidden weights and biases
Backward RNN	Its own input-to-hidden, hidden-to-hidden weights and biases
Dense	Weights and bias mapping BiRNN representations to classes

20. Training hyperparameters

```

optimizer → how weights are updated
learning_rate → size of the update steps
loss → how wrong predictions are measured
metrics → how performance is reported
batch_size → number of examples processed together
epochs → number of full passes over the training data
shuffle → whether training examples are reordered

```

A useful distinction: weights/embeddings are learned parameters; learning rate, batch size, number of units, dropout, and epochs are hyperparameters selected by you.

21. How to get the hidden representation for a particular word

If the BiRNN returns every timestep:

```

bidir = keras.layers.Bidirectional(
keras.layers.SimpleRNN(64, return_sequences=True)
)

H = bidir(embedded_x)

# H.shape:
(batch, sequence_length, 128)

# Representation for the 3rd word:
word3_repr = H[:, 2, :]

```

For each sentence, that slice contains the concatenated forward/backward representation for the third word. You can feed it into another layer, inspect it, or use it for a downstream task.

22. Padding and y: an important practical note

Real sentences do not all have the same length. Batches are commonly padded to a common length. Padding creates artificial positions, so the loss should normally avoid treating padded positions as real labels. In Keras, masking and/or an appropriate loss-weighting strategy can be used depending on the exact model and data pipeline.

This is an important next topic once the basic BiRNN shape flow is completely clear.

23. One-page mental model



```

Words / token IDs
↓
Embedding
↓
One vector per word
↓
████████████████████████████████████████
■ BiRNN ■
■ ■
■ Forward → ■
■ ← Backward ■
████████████████████████████████████████
↓
One combined representation per word
↓
Dense(5)
↓
Five class probabilities per word
↓
Compare with one integer class ID per word
↓
Loss → gradients → update weights

```

The single most important chain to remember: ``return_sequences=True`` → keep every timestep → Dense acts on each timestep → one prediction vector per word → NER-style ``y`` has one class ID per word.

24. Final cheat sheet

Concept	Remember
Batch size	How many sentences/examples are processed together.
Sequence length	How many word/timestep positions each example has.
Embedding dimension	How many numbers represent each word.
RNN units	How many hidden values each direction maintains.
BiRNN concat	Forward H + backward H = 2H features.
return_sequences=True	Keep output for every timestep.

Dense(5)	Map each timestep representation to 5 class scores/probabilities.
Token classification output	(B, T, 5) for 5 classes.
Sparse token y	(B, T) integer class IDs.
TimeDistributed	Explicit per-timestep wrapper; usually unnecessary for plain Dense here.
Sentence classification	Usually one output per sentence; return_sequences=False is common.

25. Interview / explanation version

“A Bidirectional RNN uses two RNNs, one processing the sequence left-to-right and one right-to-left. At each timestep, their hidden representations are merged, commonly by concatenation. If I use `return_sequences=True`, I retain a representation for every word. For example, with 64 units in each direction, each word gets a 128-dimensional representation. A Dense(5) layer then maps each of those representations to five class probabilities, so the output becomes `(batch, sequence_length, 5)`. With sparse categorical cross-entropy, the target `y` contains one integer class ID per word, with shape `(batch, sequence_length)`. `TimeDistributed` is not required for a plain Dense layer because Dense already applies its transformation across the final dimension of the sequence tensor.”

End of current BiRNN notes

Next natural topics: masking/padding in BiRNNs, stacked BiRNNs, LSTM/GRU bidirectionality, attention, and how BiRNN differs from Transformer encoders.